Universidad de Carabobo

Facultad de ciencias y tecnologia

Laboratorio nro.1 de Arquitectura del computador

estudiantes:

Jose Alvarez 31389258

Jose Nuñez 31581050

Introducción

El presente laboratorio tiene como objetivo principal la inmersión en los fundamentos de la programación en lenguaje ensamblador MIPS32, utilizando el simulador MARS como entorno de desarrollo. La implementación de algoritmos, tanto recursivos como iterativos, sirve como vehículo para analizar la gestión de recursos a bajo nivel, un aspecto crítico en la arquitectura de computadores.

A través de este ejercicio, se explora el comportamiendo del procesador frente a la manipulación directa de registros y la gestión dinámica de la memoria a través de la pila (**$sp**). El análisis comparativo entre ambos enfoques permitirá comprender las implicaciones de rendimiento y seguridad que conlleva la elección de una estructura algorítmica, estableciendo una base sólida para el desarrollo de software eficiente en sistemas donde el control sobre el hardware es absoluto.

1. ¿ Cómo se implementa la recursividad en MIPS32 ?

¿ Qué papel cumple la pila ($sp) ?

R-

La recursividad en MIPS se implementa mediante la técnica de auto-llamada, apoyada estrictamente en el estándar de llamadas (*Calling convention*).

- **Implementación**: Se define un *Prologue* donde se reservan espacios en la pila, se guarda la direccion de retorno (**$ra**) y los argumentos necesarios, y un *Epilogue* donde se restaura el estado previo antes de retornar (**jr $ra**).

- **Papel de la pila** (**$sp**): La pila actúa como una "memoria temporal de historial". En cada llamada recursiva, **$sp** se desplaza (decrementa en MIPS) para crear un nuevo marco (*stack frame*). Esto es vital porque el registro **$ra** solo puede almacenar una dirección a la vez; sin la pila, cada llamda nueva sobreescribiría la dirección de retorno de la anterior, dejando al programa "perdido" y sin forma de volver al flujo principal.

2. ¿ Qué riesgos de desbordamiento existen ?

¿ Cómo mitigarlos ?

R-

El riesgo principal es el **Stack Overflow** (desbordamiento de pila).

- **El riesgo**: MIPS asigna un segmento de memoria finito para la pila. Si la profundidad de la recursión es muy alta, el puntero **$sp** sigue restando direcciones hasta invadir segmentos de memoria que no le corresponden (como los datos globales o el código). Esto provoca una interrupción del programa (Runtime Exception).

- **Cómo mitigarlos**:

1- **Enfoque iterativo**: Siempre que sea posible, transformar el algoritmo recursivo en uno iterativo para utilizar registros en lugar de memoria de pila.

2- **Validacion de entrada**: Limitar el rango de entrada del usuario para asegurar que nunca supere la capacidad de profundidad de la pila.

3- **Tail Recursion** (**Recursión de cola**): Si el lenguaje o el compilador lo permiten, optimizar la función para que la llamada recursiva sea la última instrucción, permitiendo reutilizar el mismo *stack frame*.

3. ¿ Qué diferencias encontraste entre una implementación iterativa y una recursiva en cuanto al uso de memoria y registros ?

Criterio	Implementación Recursiva	Implementación Iterativa
Uso de memoria	*O(n)* (crece linealmente con *n*)	*O(1)* (constante)
Uso de registros	Intenso(guardar/cargar en cada llamada)	Mínimo (uso de registros temporales)
Flujo de control	Basado en llamadas (**jal**)	Basado en saltos(**j**, **bnez**)

En la recursiva, el costo en memoria es alto debido al almacenamiento constante de registros en la pila. En la iterativa, el estado del programa se mantiene en registros internos (como **$t0** o **$v0**), eliminando el costo de acceso a memoria ecterna.

4. ¿ Qué diferencias encontraste entre los ejemplos académicos del libro y un ejercicio completo y operativo en MIPS32 ?

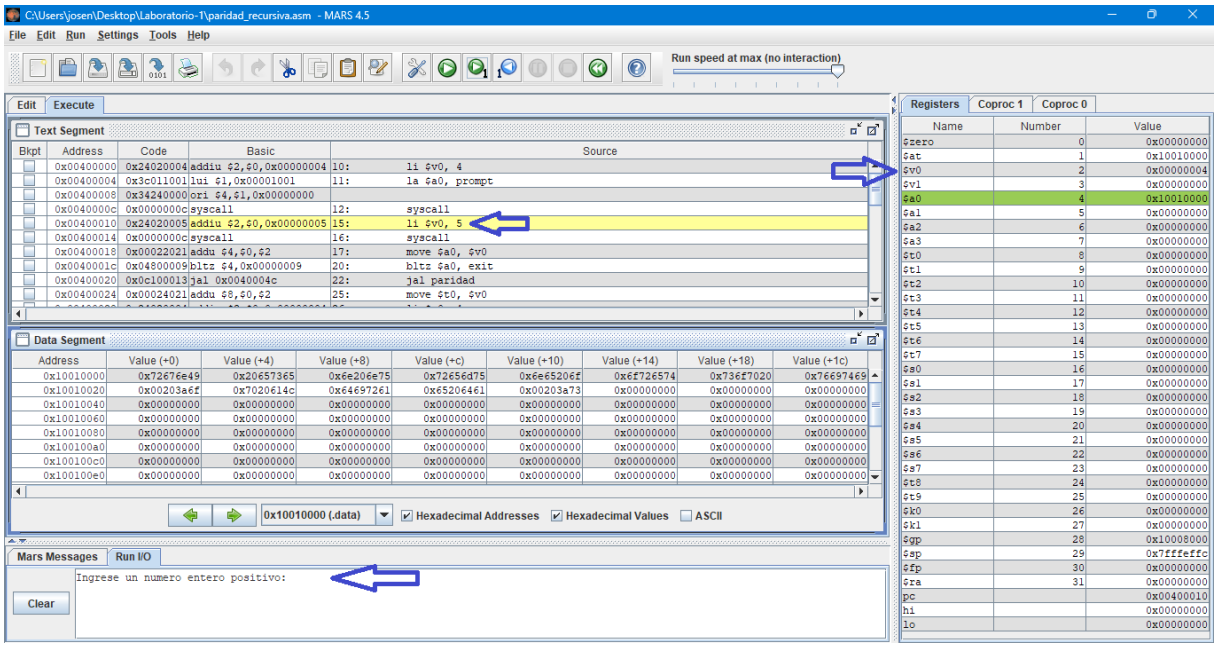
- **En la teoría**: Los libros a menudo ignoran la "crudeza" de los regitros, asumiendo que el *stack* es infinito o que el manejo de registros es automático.

- **En la práctica**: En MIPS, el programador es responsable de gestionar absolutamente todo: guardar registros que podrían ser modificados por funciones llamadas, manejar manualmente el puntero de pila y asegurar que **jal** tenga su correspondiente **jr $ra**. La diferencia es el nivel de control y, por ende, la responsabilidad de evitar errores de segmentación.

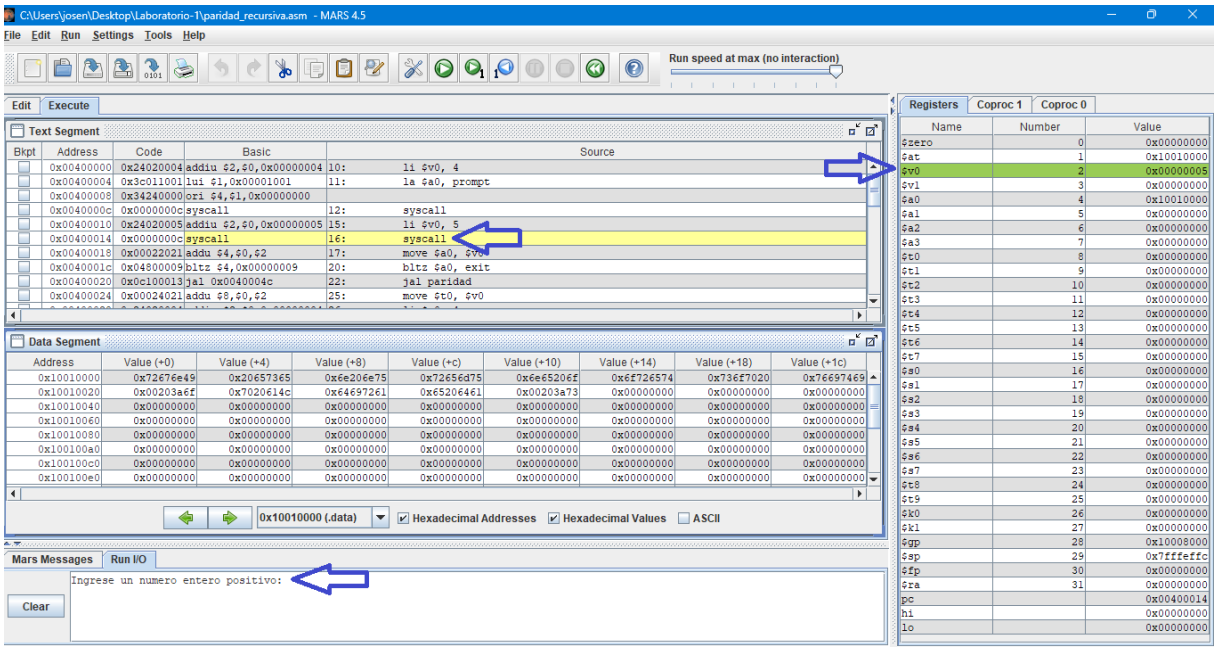
5. Elaborar un tutorial de la ejecución paso a paso en MARS

Recursivo:

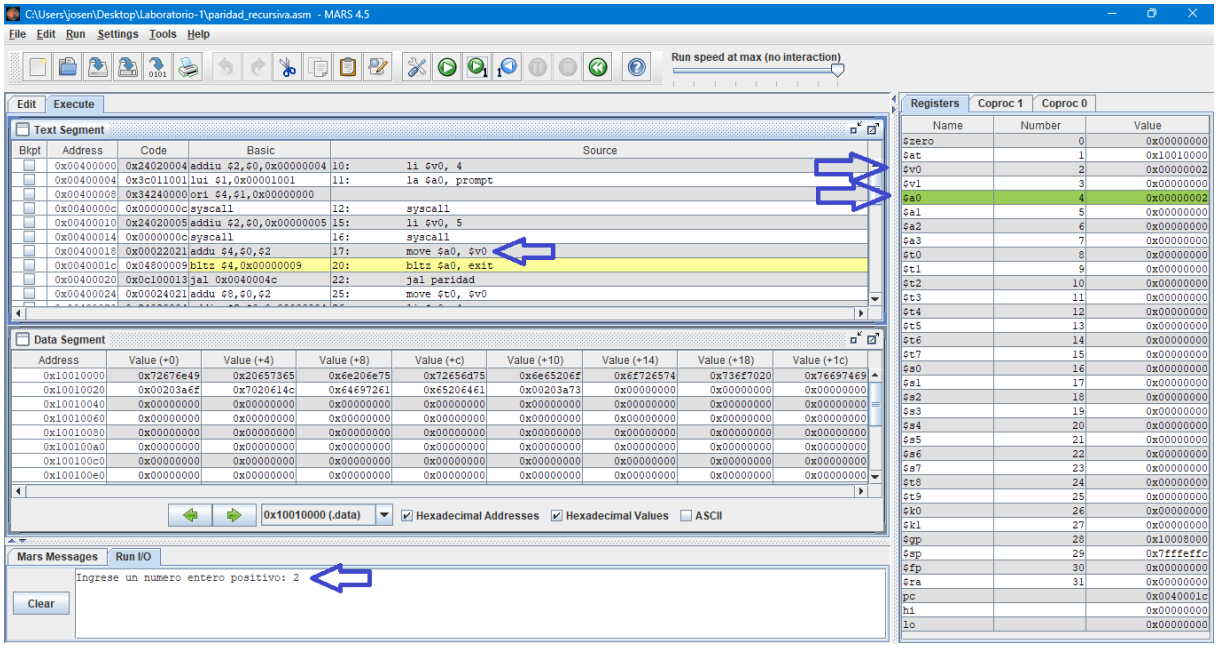
-Paso 1: Tras ensamblado el programa, este llega a la linea 10: y le asigna a $v0 el valor 4 (usado para escribir un string) para asi escribir el texto "Ingrese un numero entero positivo".



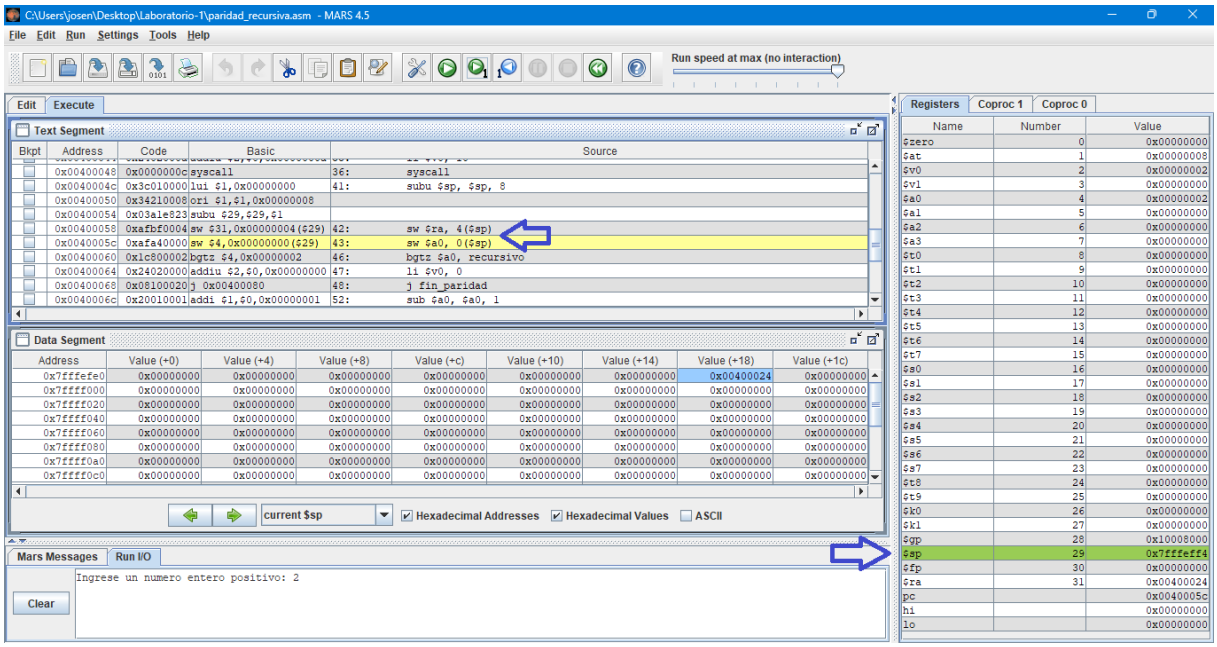
-Paso 2: Luego llega a la linea 15: y le asigna a $v0 el valor 5 (usado para recibir un entero) para asi esperar que el usuario ingrese un numero.



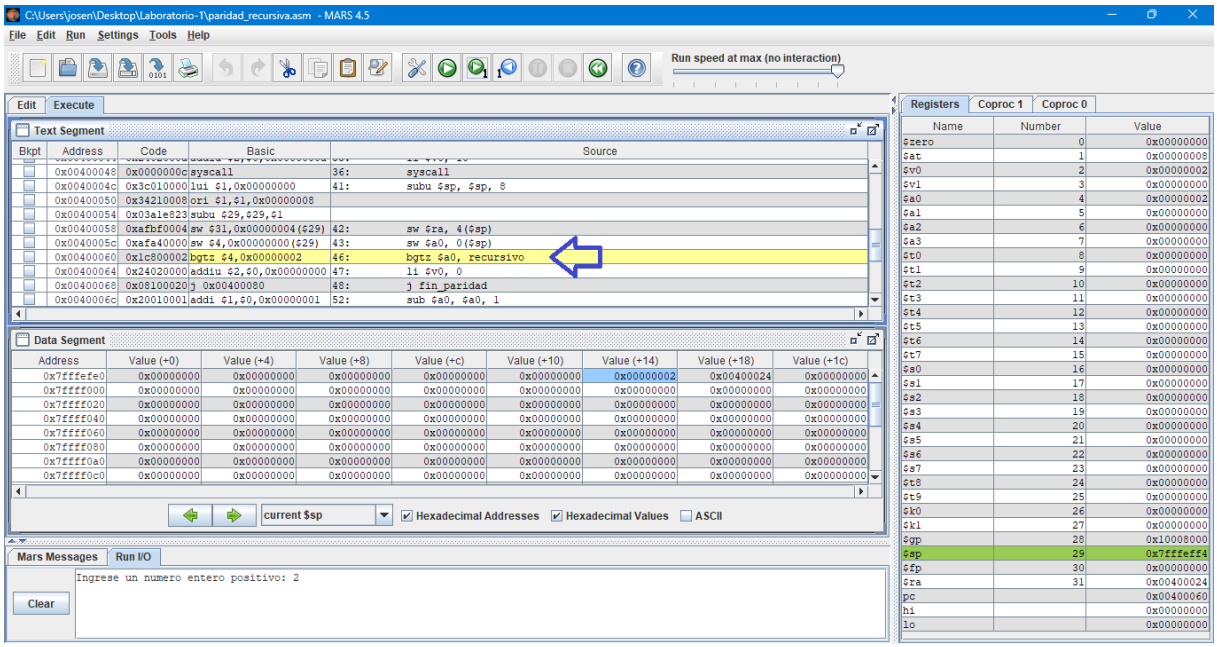
-Paso 3: Tras el usuario ingresar un valor (en este ejemplo “2”) el programa llega a la linea 17: y mueve el valor a la dirección del registro $a0.

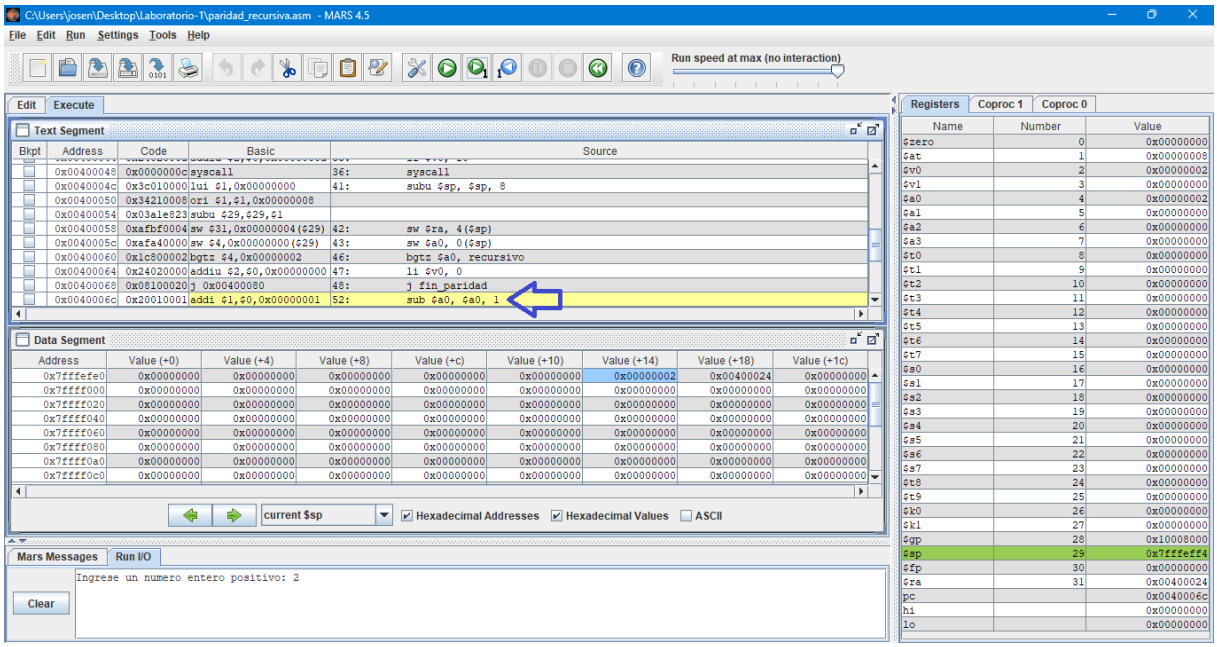


-Paso 4: El programa llega a la linea 22: y salta a la etiqueta “*paridad*”, en las lineas 41: a 43: guarda espacio en el puntero $sp para los registros $a0 (el numero n) y $ra (la memoria del paso anterior para poder regresar).

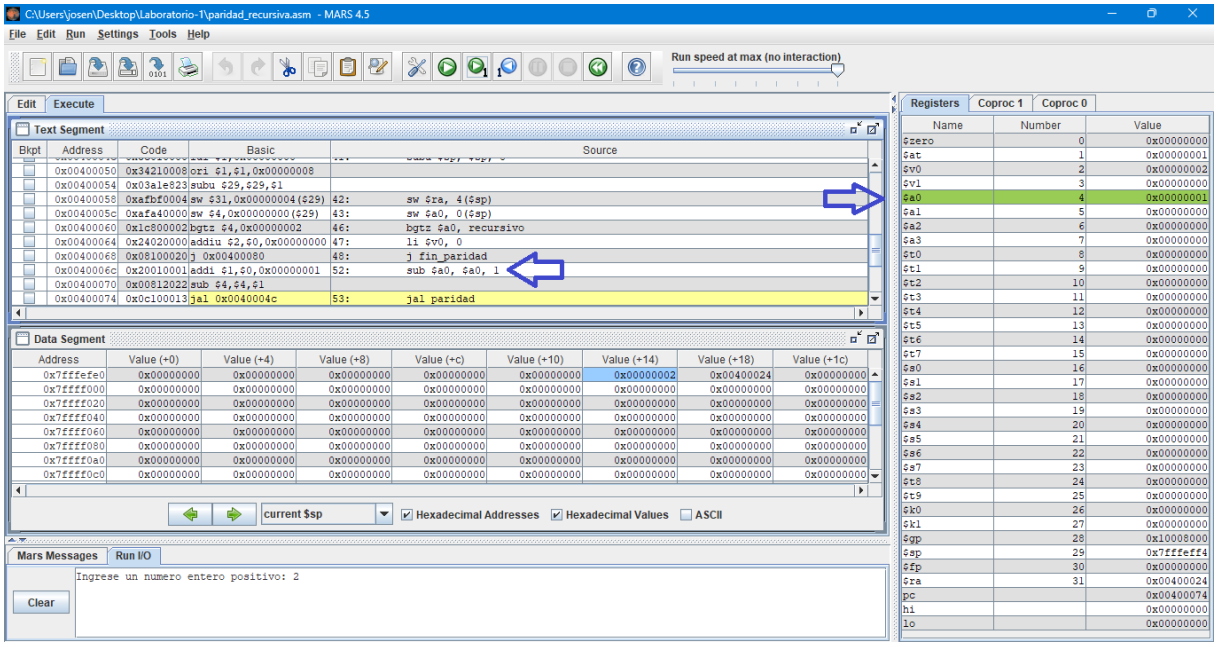


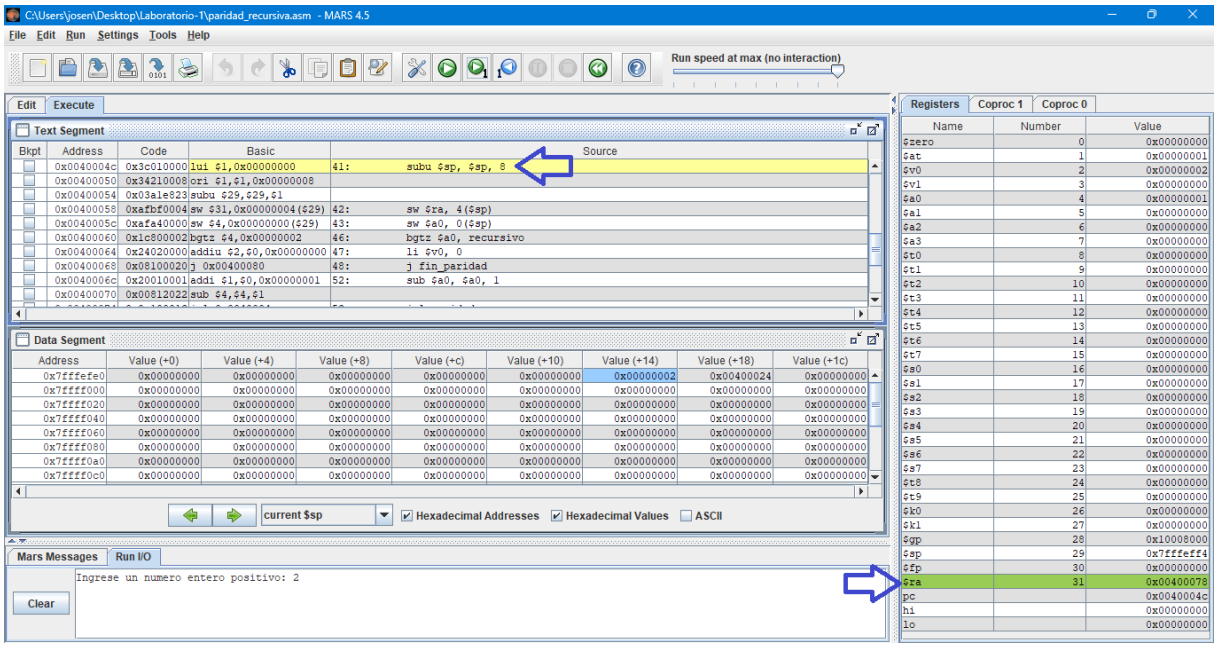
-Paso 5: El programa llega a la linea 46: y pregunta si $a0 es mayor de cero con la operacion bgtz. En este paso $a0 es igual a dos por lo tanto es verdad y salta hacia la etiqueta "*recursivo*".



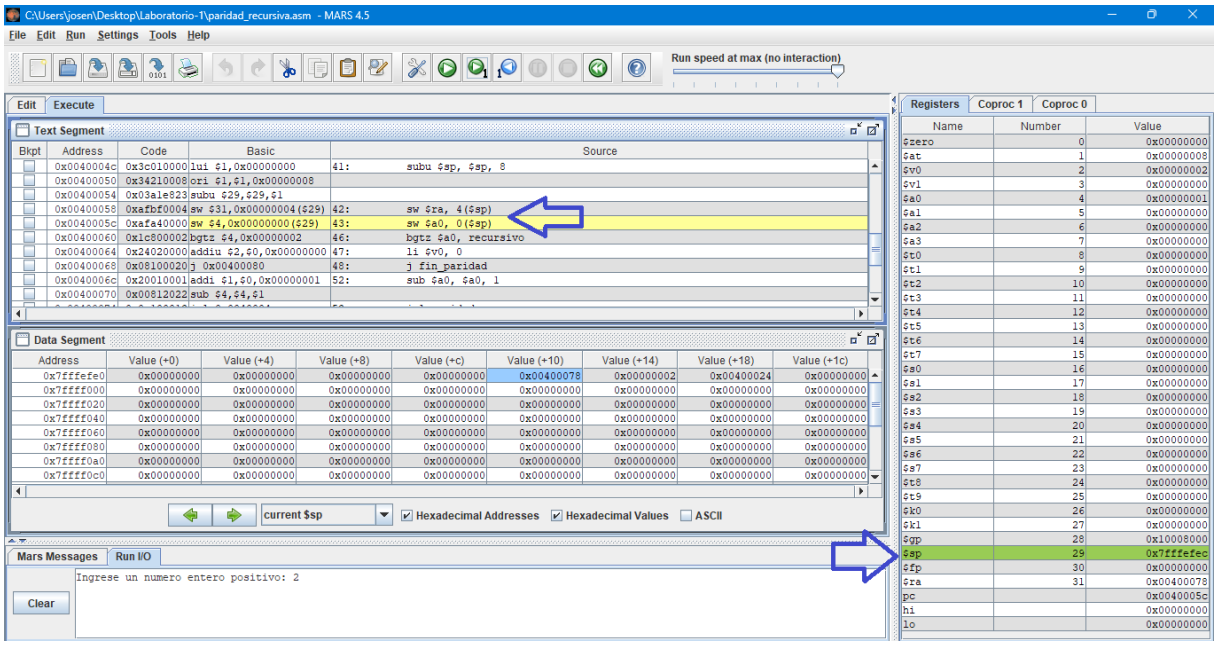


-Paso 6: El programa al llegar a la etiqueta "*recursivo*" llega a la linea 52: le resta 1 a $a0 (por lo tanto $a0 ahora es 1) y salta a la etiqueta "*paridad*".

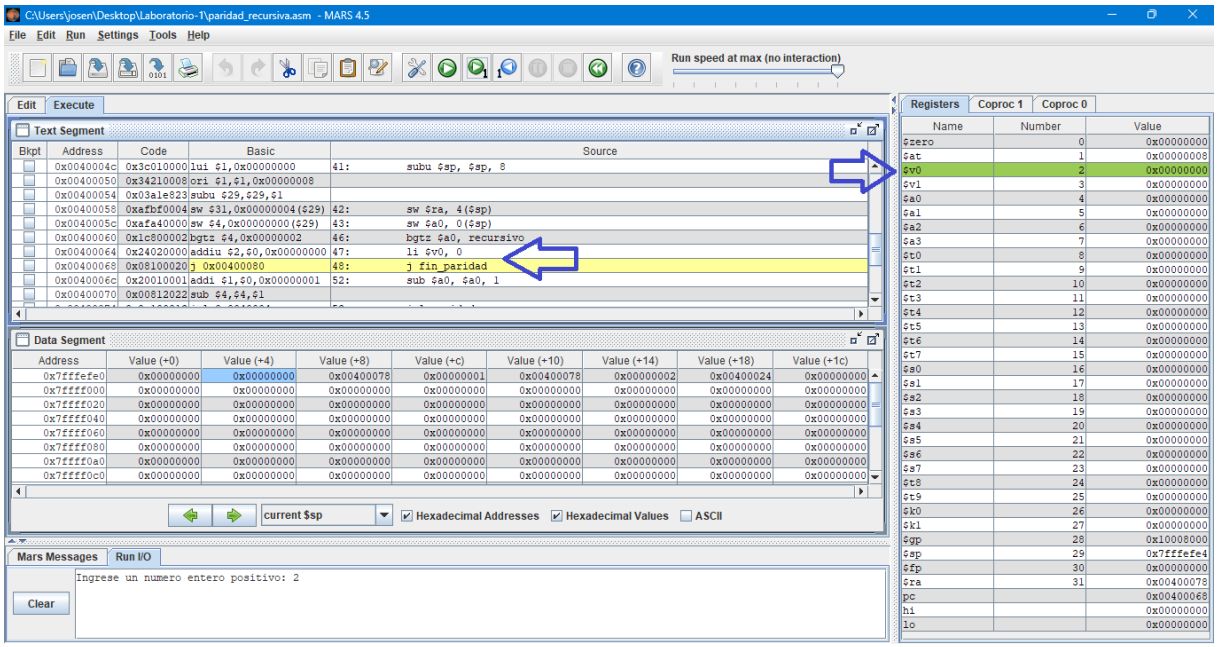




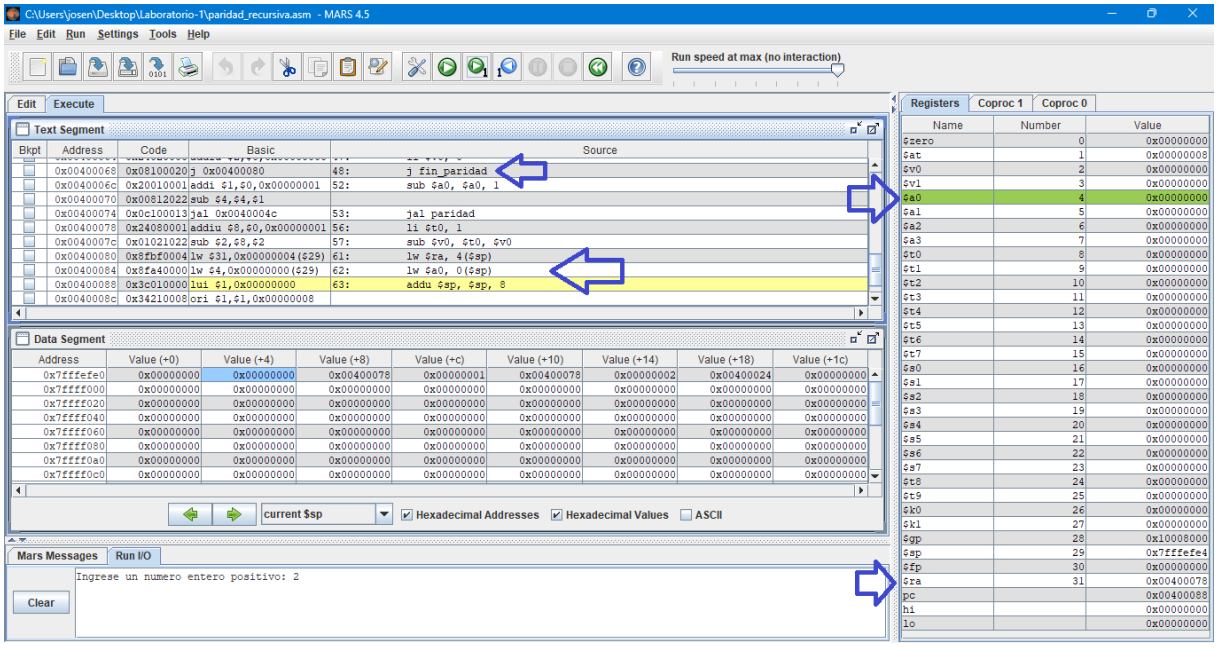
-Paso 7: Los pasos (4), (5) y (6) se repiten hasta que $a0 sea igual a cero. Por cada paso el valor del puntero $sp va cambiando, lo que significa que se esta usando otra "caja" de memoria.

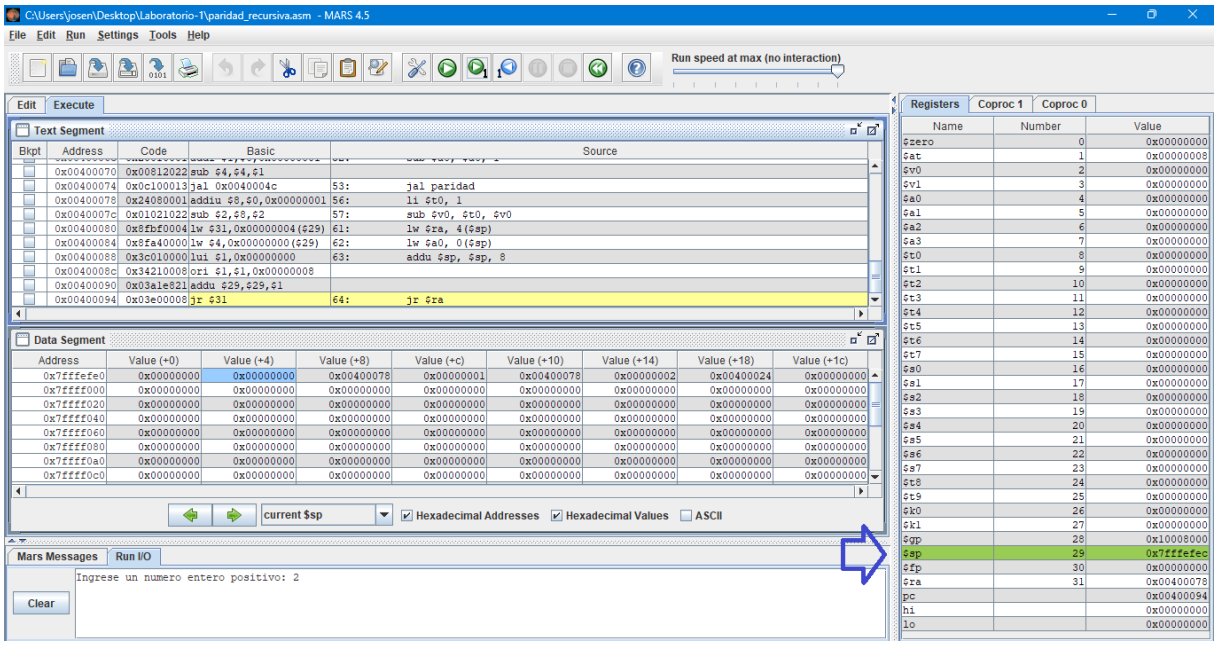


-paso 8: Tras varias pasadas, el valor del registro $a0 ahora es cero y por lo tanto la linea 46: (la de operación bgtz) da falso y no salta hacia la etiqueta "*paridad*", en cambio sigue por las lineas 47: y 48:.

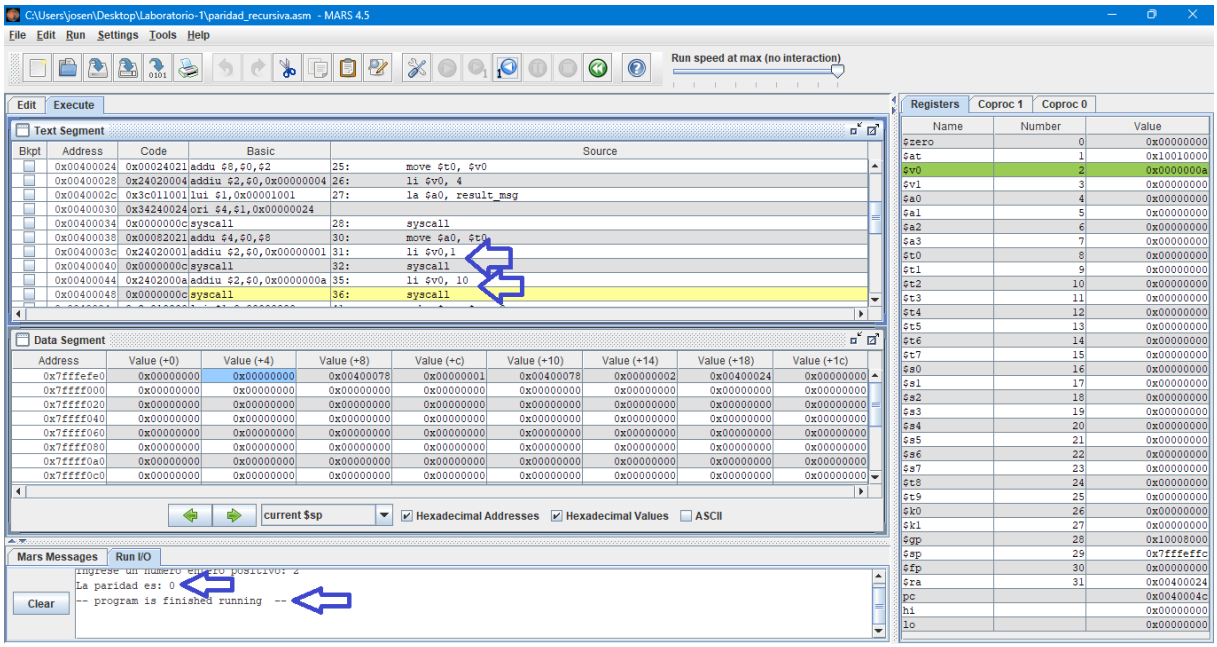


-Paso 9: El programa al llegar a la linea 48: salta hacia la etiqueta "*fin_paridad*" en donde se recupera el valor de puntero $sp para ir regresando a la "caja" anterior, así hasta regresar a la "caja" original (el main).



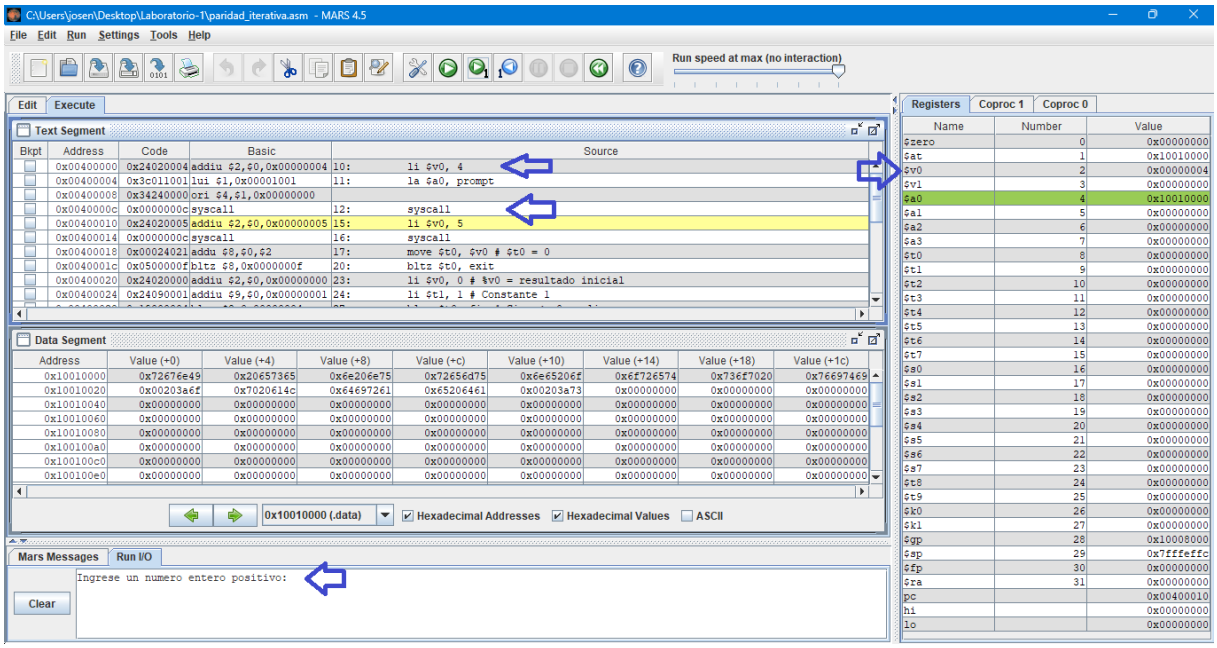


-Paso 10: El programa cuando regresa al main pasa por las lineas 25: a 28:, le asigna al registro $v0 el valor 4 para asi escribir el mensaje "*La paridad es:*". Luego le asigna al registro $v0 el valor 1 (usado para escribir un entero) y escribe el valor resultante.

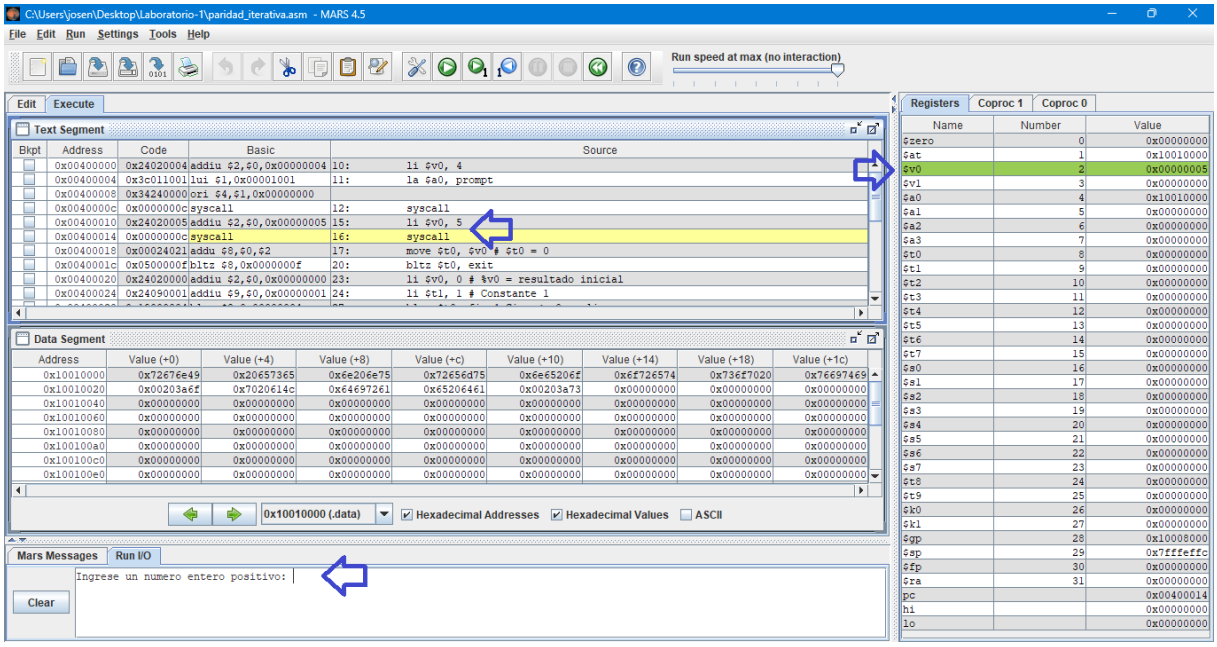


Iterativo:

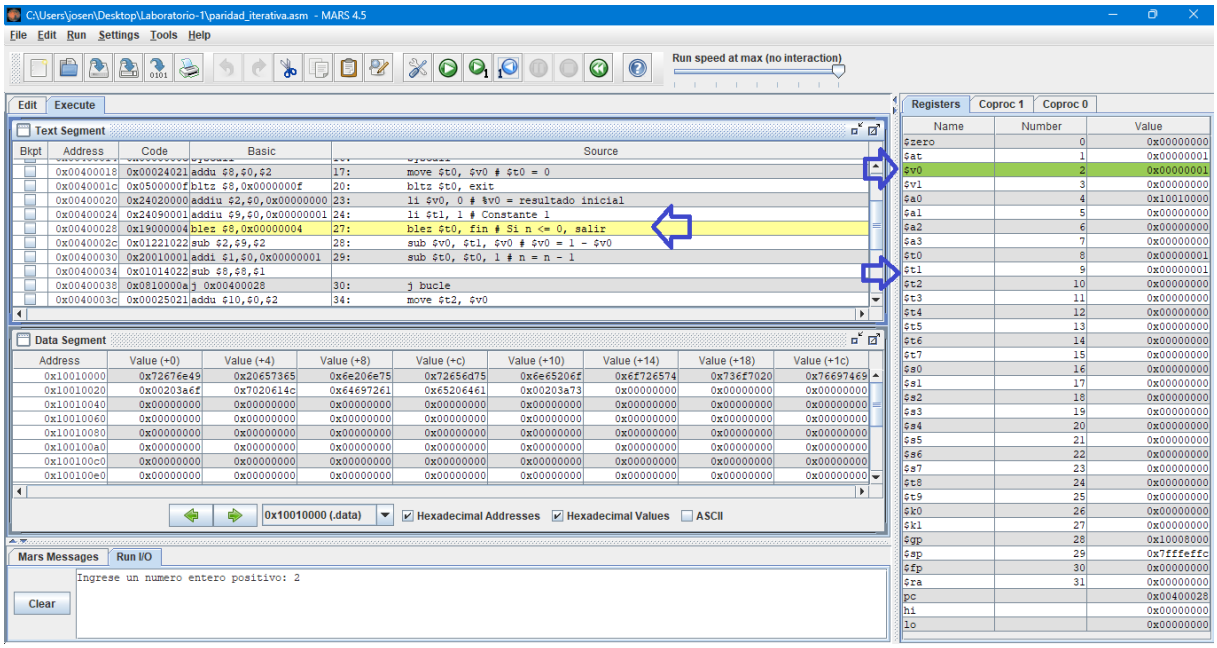
-Paso 1: Tras ensamblado el programa, este llega a la linea 10: y le asigna a $v0 el valor 4 (usado para escribir un string) para asi escribir el texto "Ingrese un numero entero positivo".



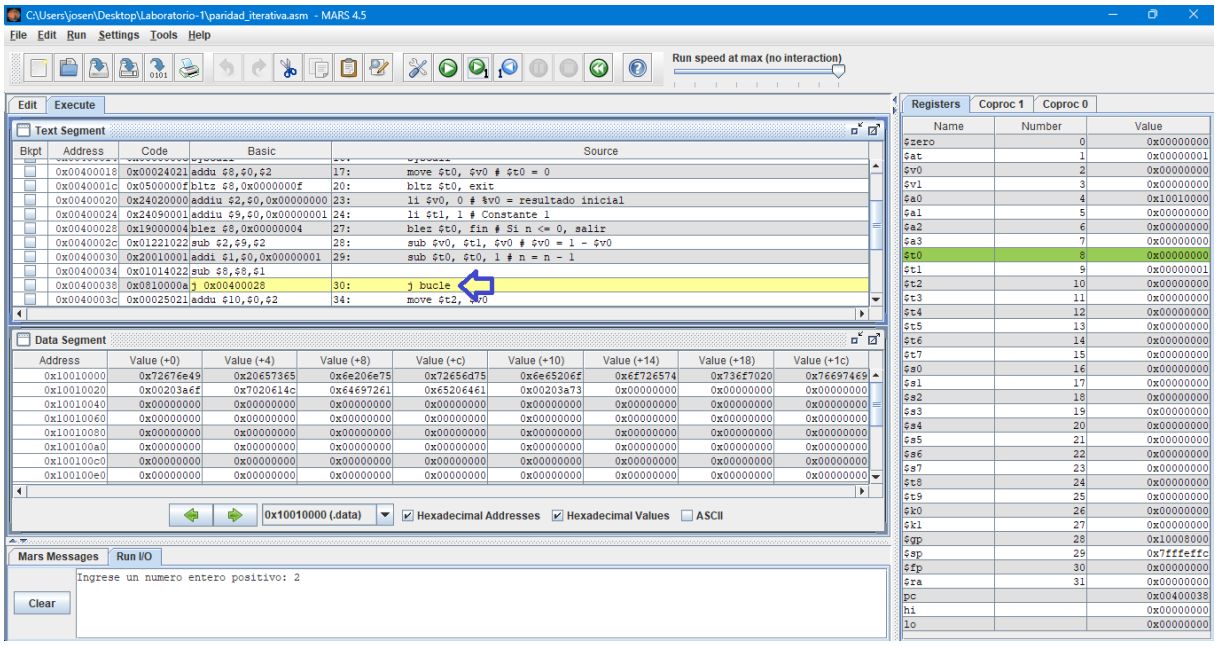
-Paso 2: Luego llega a la linea 15: y le asigna a $v0 el valor 5 (usado para recibir un entero) para asi esperar que el usuario ingrese un numero.

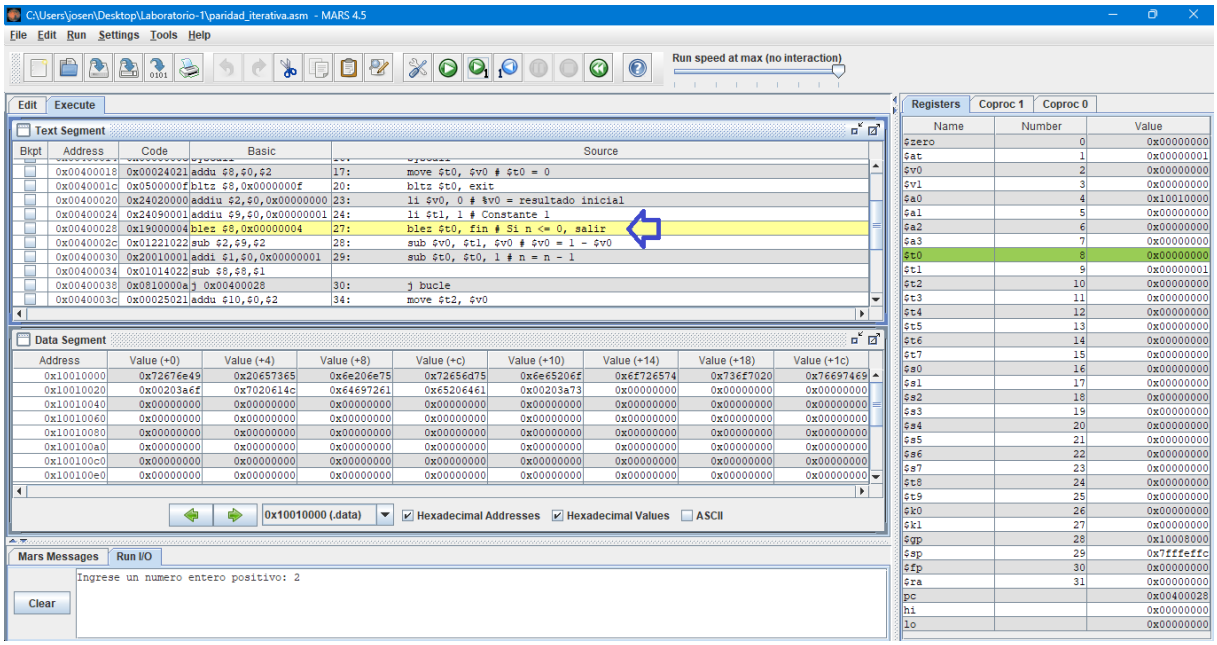


-Paso 3: Tras recibir el valor, el programa llega a la linea 27: donde usa la funcion blez para saber si el valor n ingresado ya llego a cero. Si todavia no llego a cero sigue a las lineas 28: y 29: en donde se le resta uno al valor n (n-1).

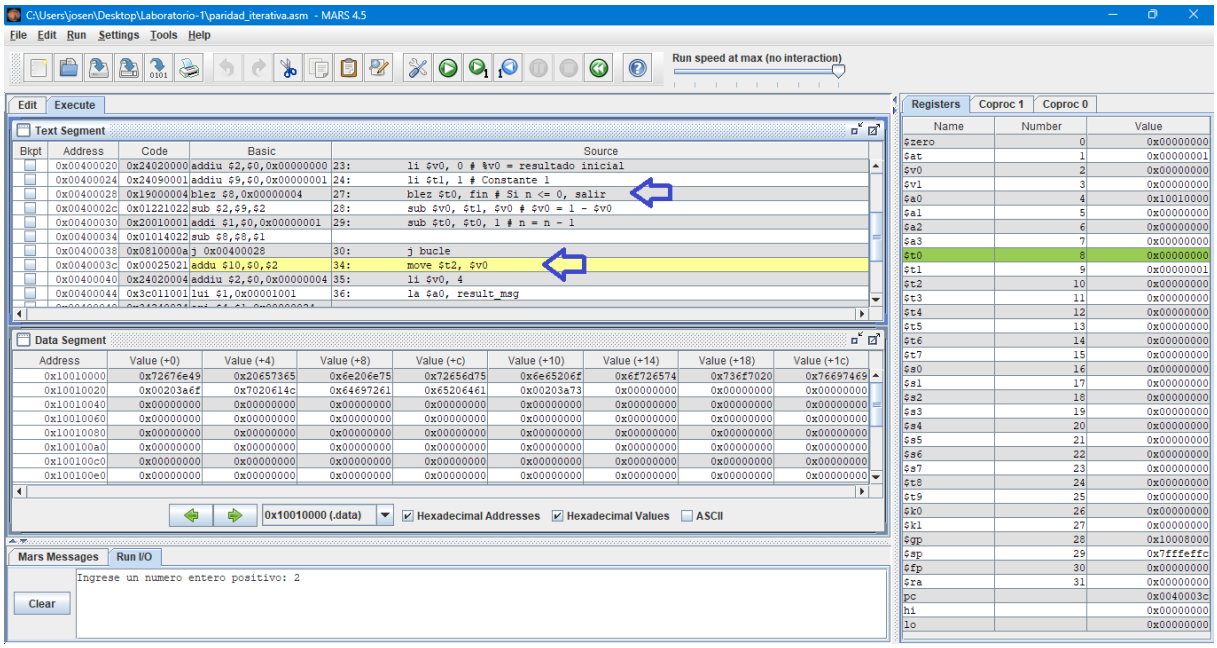


-Paso 4: El programa llega a la linea 30: en donde se aplica el salto hacia la etiqueta “bucle” (j bucle). Los pasos (3) y (4) se repiten hasta que el valor n se vuelva cero.

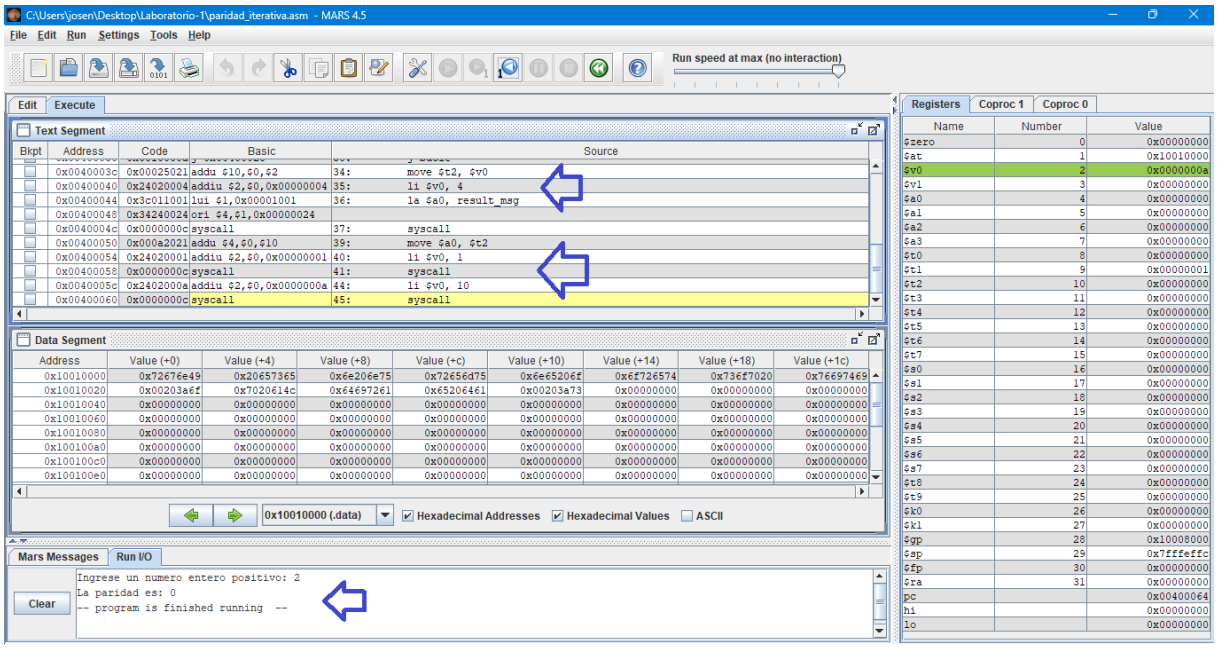




-Paso 5: Tras que se repita el paso (4) hasta que n se vuelva cero, la linea 27: de funcion blez es verdad y entonces salta hacia la etiqueta “salir”.



-Paso 6: Tras saltar a la etiqueta “salir” llega a las lineas 34: a 37: en donde se prepara el mensaje “La paridad es: ”. Luego finalmente llega a las lineas 39: a 45: en donde retorna el valor resultante.



6. Justificar la elección del enfoque (iterativo vs. recursivo) según eficiencia y claridad

- **Eficiencia** (**Ganador: Iterativo**): Es indiscutiblemente más eficiente. No requiere el *overhead* de gestión (push/pop) ni el riesgo de desbordamiento. Para sistemas embebidos, donde la memoria es un recurso crítico, el enfoque iterativo es obligatorio.

- **Claridad** (**Ganador: Recursivo**): La recursividad brilla en la legibilidad. Si miras la definición matemática del problema:

paridad(n) = 1 − paridad(n − 1)

El código recursivo es una traduccion casi literal de la fórmula. Es mucho más facil de auditar y mantener para problemas matemáticos complejos o estructuras de datos (como árboles).

Conclusión técnica: Elegir uno obre otro es un *trade-off*. En entornos de producción con restricciones de memoria, la iteración es el estándar. En entornos de prototipado donde la elegancia y la rapidez de desarrollo son prioridad, la recursión es una herramienta poderosa.

7. Análisis y discusión de los resultados

- **Sobre el entorno MARS**: El simulador nos permite ver el proceso que toma el procesador al ejecutor los programas (lo que nos ayudo a entender mejor el **MIPS32**).

- **Resultados observados**: Al probar con valores pequeños, ambos enfoques entregaron los mismos resultados correctos. Sin embargo, al monitorear el registro **$sp** en la versión recursiva, fue evidente el crecimiento de la pila, confirmando que la recursión es una herramienta poderosa pero costosa en términos de memoria.

- **Aprendizaje principal**: Como dicho antes, el usar el entorno MARS para desarrollar y ejecutar los programas y el ver como se comportan los registros y pilas a nivel del procesador nos permitio entender un poco mejor el como funciona el lenguaje de alto nivel, su idea y el como se puede usar de mejor manera.

Conclusión

Tras finalizar el desarrollo y la experimentación en MARS, se concluye que la elección entre una solución iterativa y una recursiva no es meramente una cuestión de estilo, sino una decisión de ingeniería condicionada por las restricciones del sistema.

Si bien la recursividad ofrece una expresividad matemática superior y una implementación más cercana a la definición lógica del problema, su costo en memoria y el riesgo inherente de *stack overflow* la posicionan como una herramienta de uso cuidadoso. En contraparte, la implementación iterativa demuestra ser la opción robusta y eficiente al mantener un uso constante de la memoria (***O(1)***), lo que garantiza la estabilidad en la ejecución, independientemente de la magnitud de la entrada.

En última instancia, este laboratorio subraya la importancia de la convención de llamadas (*calling convention*) y el papel vital que desempeñan los registros y la pila en la ejecución de cualquier programa. El dominio de estos conceptos es, sin duda, la diferencia entre un código funcional y uno optimizado, capaz de operar de manera eficiente en la arquitectura subyacente.